

Week 14

Training Neural Networks in Practice

The Features Matter More Than the Classifier

Section	Question it answers
1. The Temporal Dataset	What information were we throwing away for 13 weeks?
2. Features Matter More Than the Classifier	Why did logistic regression beat the MLP?
3. Practical Training	What happens inside MLPClassifier?
4. What Goes Wrong	How do you diagnose training failures?
5. Data Augmentation	Can we make 480 trials behave like 4,800?
6. Interpreting What the Network Learned	Which neurons and time bins matter?
7. Drift Robustness	Do temporal features survive electrode drift?
8. The Decision Framework	Given a new dataset, how do you choose a method?

1. The Temporal Dataset

What information were we throwing away for 13 weeks?

Week 13 opened with an argument: mean firing rates discard everything about temporal structure — when a neuron starts firing, whether it ramps or bursts, whether impairment manifests as delayed onset rather than a change in average rate. This week we test that argument.

We add temporal neural data to the dataset: 80 neurons $\times$ 50 time bins (10 ms bins over 500 ms) = 4,000 features per trial. Instead of a single mean rate per neuron, we now have the full temporal response: baseline, ramp-up, peak, and decay. Crucially, impaired subjects show a subtle delayed onset and slower decay that the mean firing rate averages away.

Figure 1 shows the difference. Panel A: a single trial from one neuron — the healthy subject's response peaks earlier and decays faster. Panel B: the trial average with standard deviation shading makes the ~ 40 ms peak delay clear. Panel C: when we collapse the temporal profile to a single mean rate (the features we used in Weeks 5–13), the healthy and impaired bars are nearly identical. The timing difference is invisible.

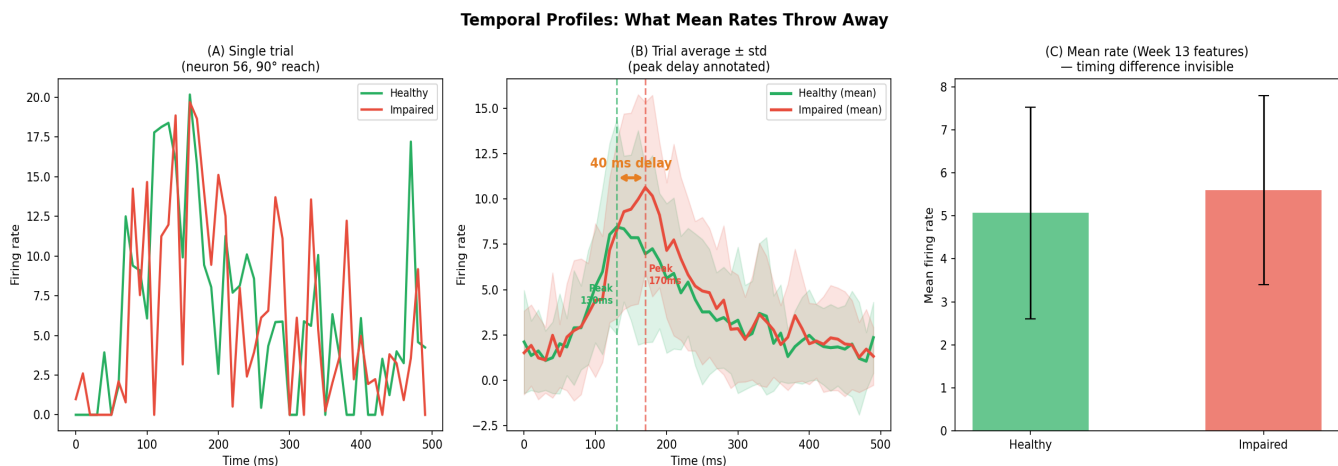


Figure 1. Temporal profiles: what mean rates throw away. (A) Single trial: healthy (green) peaks earlier than impaired (red). (B) Trial averages reveal a ~ 40 ms peak delay. (C) Mean firing rate collapses this difference — the bars are nearly identical.

The two approaches differ in a single step. For each neuron n on each trial, we record its firing rate in 50 time bins: $\text{rate}_n(t_1), \text{rate}_n(t_2), \dots, \text{rate}_n(t_{50})$. The Week 13 pipeline averaged them:

Mean rate feature: $\text{mean}_n = (\text{rate}_n(t_1) + \text{rate}_n(t_2) + \dots + \text{rate}_n(t_{50})) / 50$

One number per neuron, 80 neurons $\rightarrow$ 80 features. Everything about when the neuron fired is lost. This week's pipeline keeps them all:

Temporal features: $\text{rate}_1(t_1), \text{rate}_1(t_2), \dots, \text{rate}_1(t_{50}), \text{rate}_2(t_1), \dots, \text{rate}_{80}(t_{50})$

All 50 bins for all 80 neurons $\rightarrow$ 4,000 features. The onset delay, the peak timing, and the decay rate are all preserved. Figure 2 shows how these two representations flow through the same MLP with very different results.

Same Network, Different Features

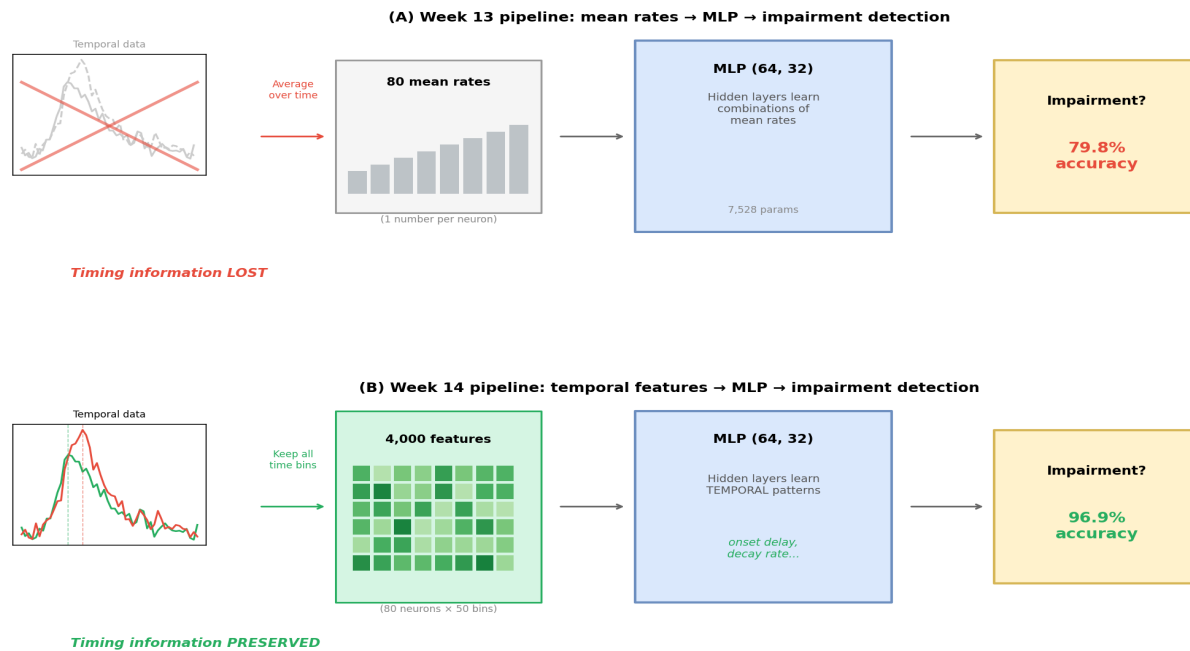


Figure 2. Two pipelines, same network, different features. (A) Week 13: temporal data is averaged to 80 mean rates before the MLP sees it — timing information lost, 79.8% accuracy. (B) Week 14: all 4,000 temporal features are preserved — the MLP can learn onset delay and decay patterns, 96.9% accuracy.

Figure 3 tells two very different stories. Panel A: for direction decoding, temporal features add nothing — all methods stay at ~95%, with changes of just ± 1 –2 points. Cosine tuning is fully captured by mean rates; extra time bins are noise. Panel B: for impairment detection, the transformation is dramatic. LR jumps 26 points, MLP jumps 17, and even LDA gains 16. The timing information that mean rates discarded was exactly what distinguished healthy from impaired.

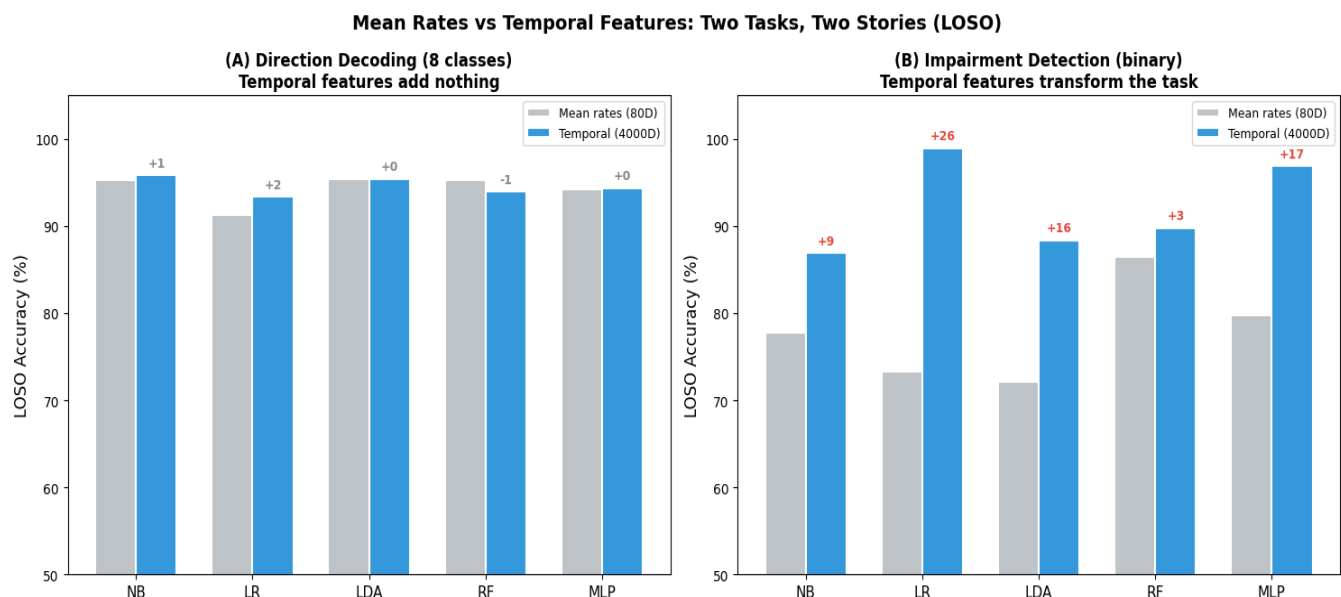


Figure 3. Mean rates vs temporal features on two tasks. (A) Direction decoding: temporal features add nothing — cosine tuning is already captured by mean rates. (B) Impairment detection: every method improves. LR +26, MLP +17, LDA +16, RF +3, NB +9. The impairment signal lives in the timing.

This is the payoff that Week 13 predicted: the impairment signal lived in the timing, not the average rate. For 13 weeks, we threw away exactly the information we needed for impairment detection.

2. Features Matter More Than the Classifier

Why did logistic regression beat the MLP?

The headline result from Figure 3 contains a surprise: LR at 99.0% beat the MLP at 96.9%. The simpler model won. Does this contradict everything Week 13 said about the power of hidden layers?

No — it illustrates a deeper lesson. In 4,000 dimensions, LR has 4,001 parameters (one weight per time bin per neuron, plus one bias). It can directly assign a large weight to the time bins where healthy and impaired subjects diverge — the onset window around 50–100 ms. The impairment signal is linearly separable in 4,000 dimensions: you just need to weight the right time bins, and LR does exactly that.

The MLP's hidden layers are unnecessary for a linearly separable problem. Its extra parameters (7,528 vs 4,001) create a slight overfitting risk without adding predictive power. This is the same pattern from Week 13: LDA beat MLP on mean rates because the 8-direction task was (nearly) linearly separable in 80 dimensions.

Meanwhile, RF gained only 3 points. Why? Random forests make axis-aligned splits — each tree asks “is time bin 47 of neuron 12 above or below threshold?” In 4,000 dimensions with 480 trials, finding the right bins and thresholds is like searching for a needle in a haystack. NB gained just 9 points because its independence assumption fails catastrophically with 4,000 correlated time bins.

Figure 4 confirms where LR is looking. The absolute LR coefficients, averaged across neurons, peak in the onset window (40–120 ms) — exactly where healthy and impaired temporal profiles diverge (see Figure 1B).

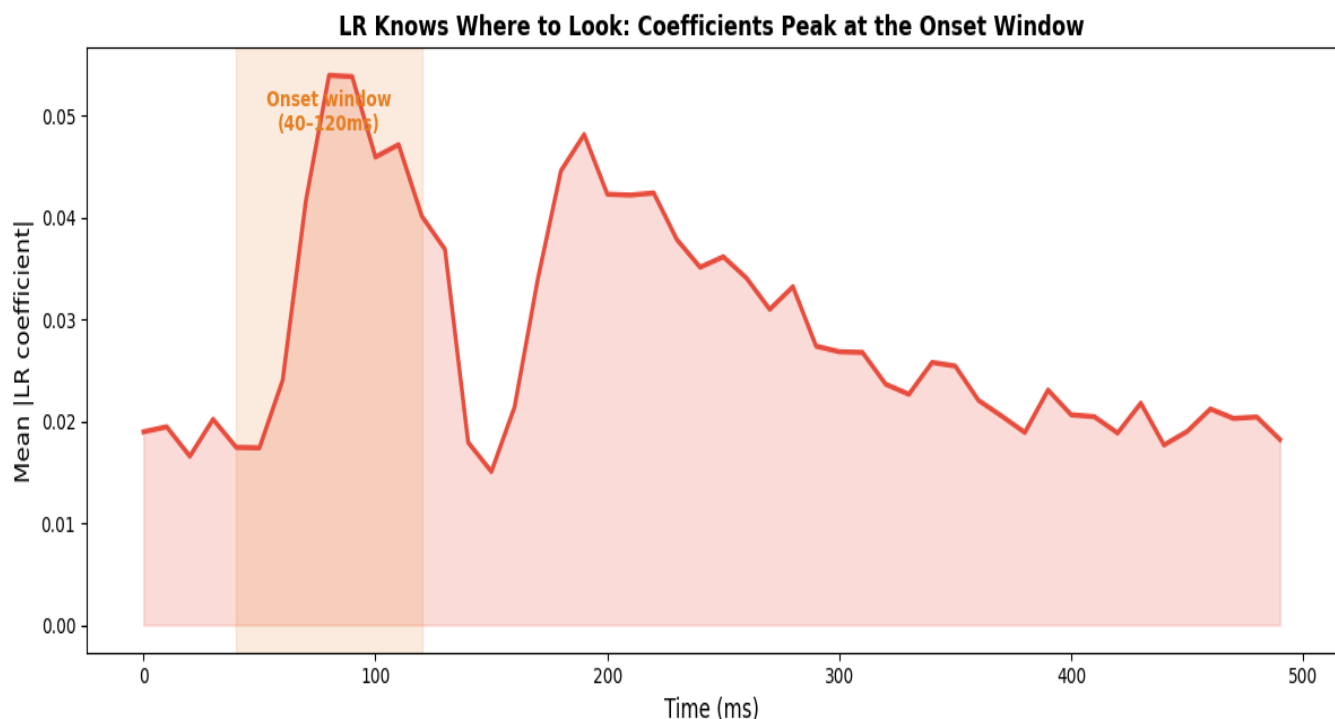


Figure 4. LR knows where to look. Absolute coefficients averaged across 80 neurons peak in the onset window (40–120 ms, highlighted), where healthy subjects ramp up earlier than impaired. LR's interpretability extends naturally to temporal features.

The meta-lesson: the features matter more than the classifier. Giving LR temporal features (99.0%) was worth more than giving RF sophisticated splits on mean rates (86.5%). Before trying a fancier model, try richer features.

3. Practical Training — From sklearn to PyTorch

What happens inside MLPClassifier?

sklearn's MLPClassifier hides the training process behind `.fit()`. That was fine for Week 13, where the goal was understanding concepts. But to train effectively on real data — especially high-dimensional temporal data — you need to control the training loop yourself.

PyTorch is a deep learning library that gives you this control. What can it do that sklearn cannot?

- Custom architectures. sklearn only offers fully connected layers. PyTorch lets you build any architecture: shared layers between tasks, skip connections, attention mechanisms — structures that the Week 15 handout will describe.
- GPU acceleration. sklearn runs on CPU only. PyTorch can train on GPUs, which becomes essential when datasets grow from 480 trials to tens of thousands.
- Fine-grained training control. sklearn gives you one learning rate for the entire training. PyTorch lets you schedule the learning rate (start high, decay over time), apply different rates to different layers, and implement custom early stopping logic.
- Gradient access. sklearn does not expose gradients. PyTorch gives you the gradient of any output with respect to any input — this is how we compute the saliency maps in Section 6.

For our 480-trial dataset, sklearn is perfectly adequate — and faster to write. But the moment you need any of the above, PyTorch becomes necessary. The lab will write a PyTorch training loop that does the same thing sklearn does, but explicitly:

- Define the model: same (64, 32) architecture as sklearn, written as a PyTorch Module with two linear layers and relu activations.
- Choose the loss: cross-entropy — the same function from Week 5 (logistic regression) and Week 13 (backpropagation).
- Choose the optimizer: Adam — the “steering wheel” teased in Week 13. It adapts the learning rate for each weight automatically.
- Loop: for each epoch, forward pass → loss → backward pass → `optimizer.step()`. This is Figure 11 from Week 13, now written in code.

Figure 5 shows the training and validation loss curves from the PyTorch training loop on our temporal data. The validation accuracy peaks at epoch 20 (98.6%), then training continues to overfit. Early stopping would halt training at this point — a technique we discussed in Week 13 that now becomes essential on 4,000 features.

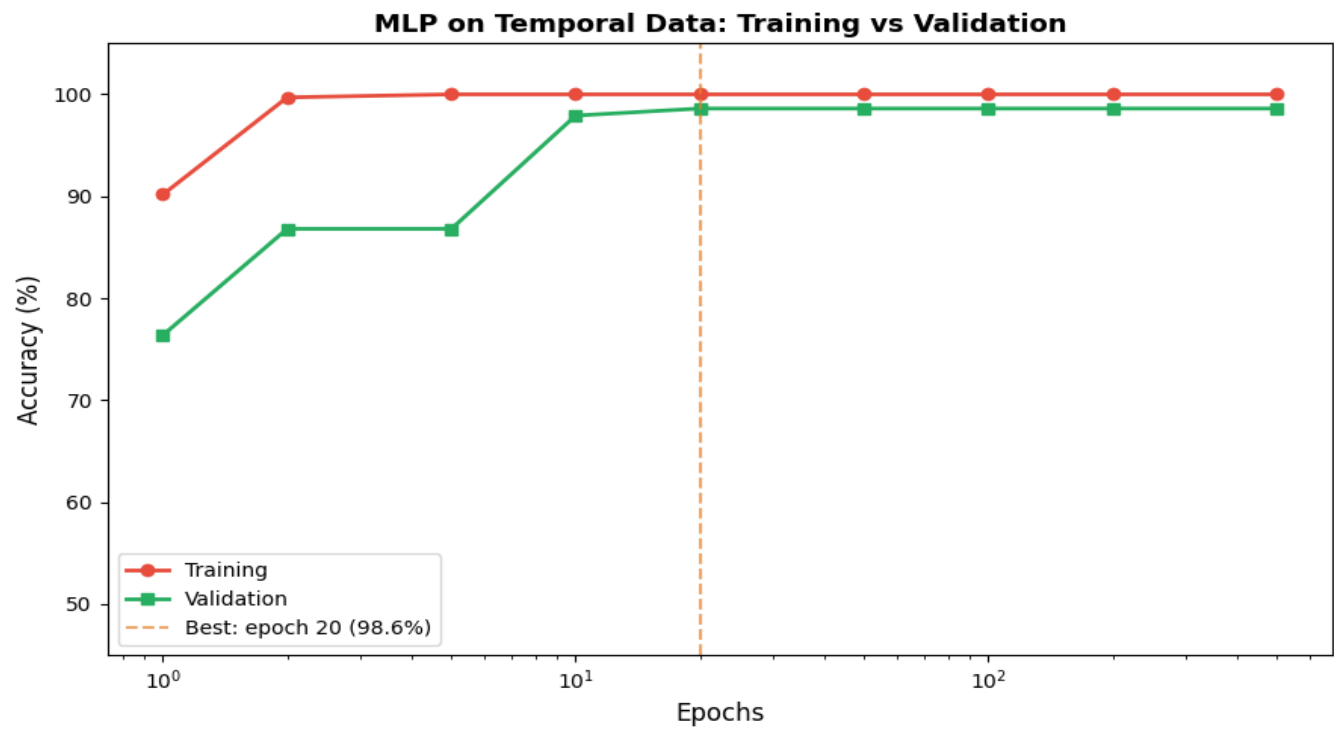


Figure 5. Training vs validation accuracy on temporal impairment detection. Validation peaks at epoch 20 (dashed line). Beyond this, the network memorises the training data without improving on held-out trials.

4. What Goes Wrong — Diagnosing Training Failures

How do you know when training is failing?

The single most practical skill in this lecture is learning to read loss curves. When you train a neural network on your own data, the loss curve is your first diagnostic tool — it tells you whether training is working before you ever look at accuracy. Here is a practical checklist to follow every time you train:

Step 1: Plot the loss curve after every training run. Not accuracy — loss. The loss is what the optimizer actually minimises, and its shape reveals problems that accuracy can hide (e.g., the loss may oscillate wildly while accuracy stays near 50%, giving no useful feedback).

Step 2: Check for three failure patterns. Each has a characteristic signature, a diagnosis, and a fix.

Failure 1: Learning rate too high. Figure 6 shows what happens when we set the learning rate to 0.1 on our temporal data. The loss zigzags wildly — each gradient step overshoots the minimum and bounces to the other side. The network never settles into a good solution. The symptom is unmistakable: a jagged, non-decreasing loss curve. The fix is simple: reduce the learning rate by 10× or 100× (e.g., from 0.1 to 0.001).

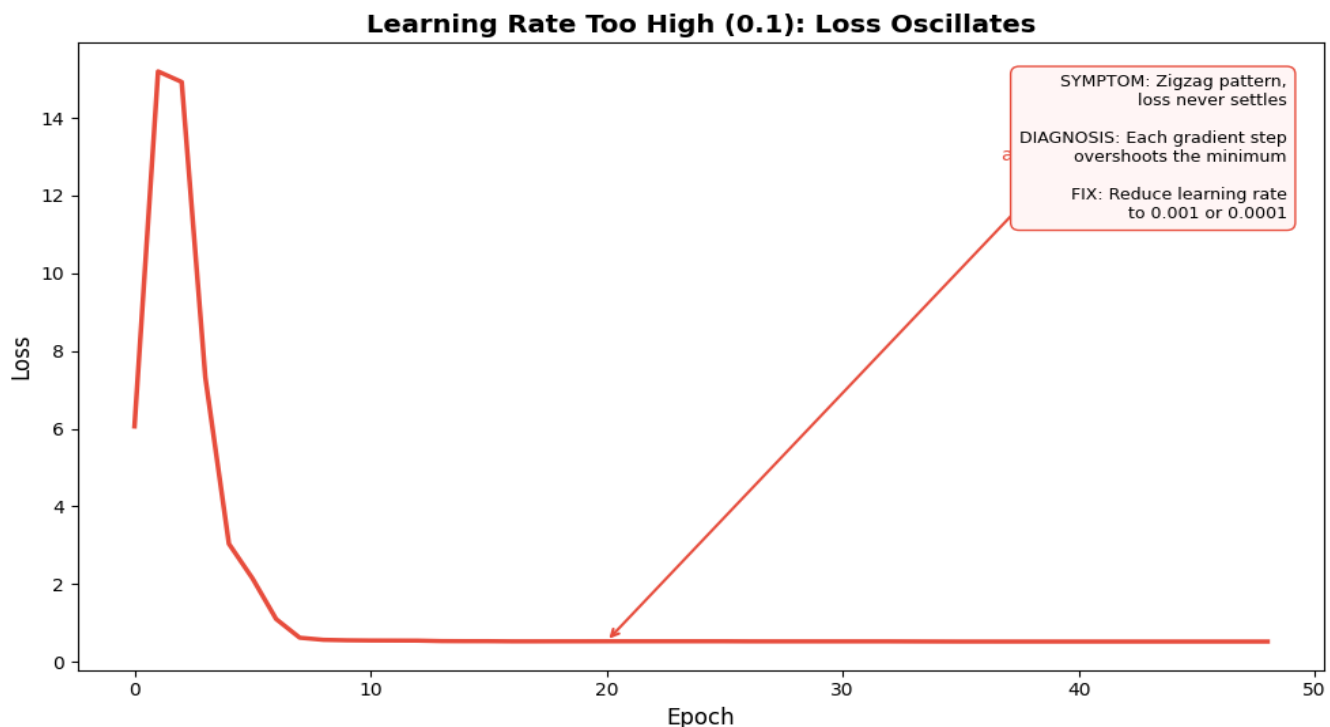


Figure 6. Learning rate too high (0.1). The loss oscillates wildly because each gradient step overshoots the minimum. The diagnostic box summarises the symptom, diagnosis, and fix.

Failure 2: Learning rate too low. Figure 7 shows the opposite problem: a learning rate of 0.00001. The loss barely decreases after 200 epochs — the network is taking steps so small that it cannot reach a good solution in reasonable time. The green dashed line shows where a well-tuned learning rate (0.001) would be at the same epoch — far lower loss. The fix: increase the learning rate by 10× to 100×.

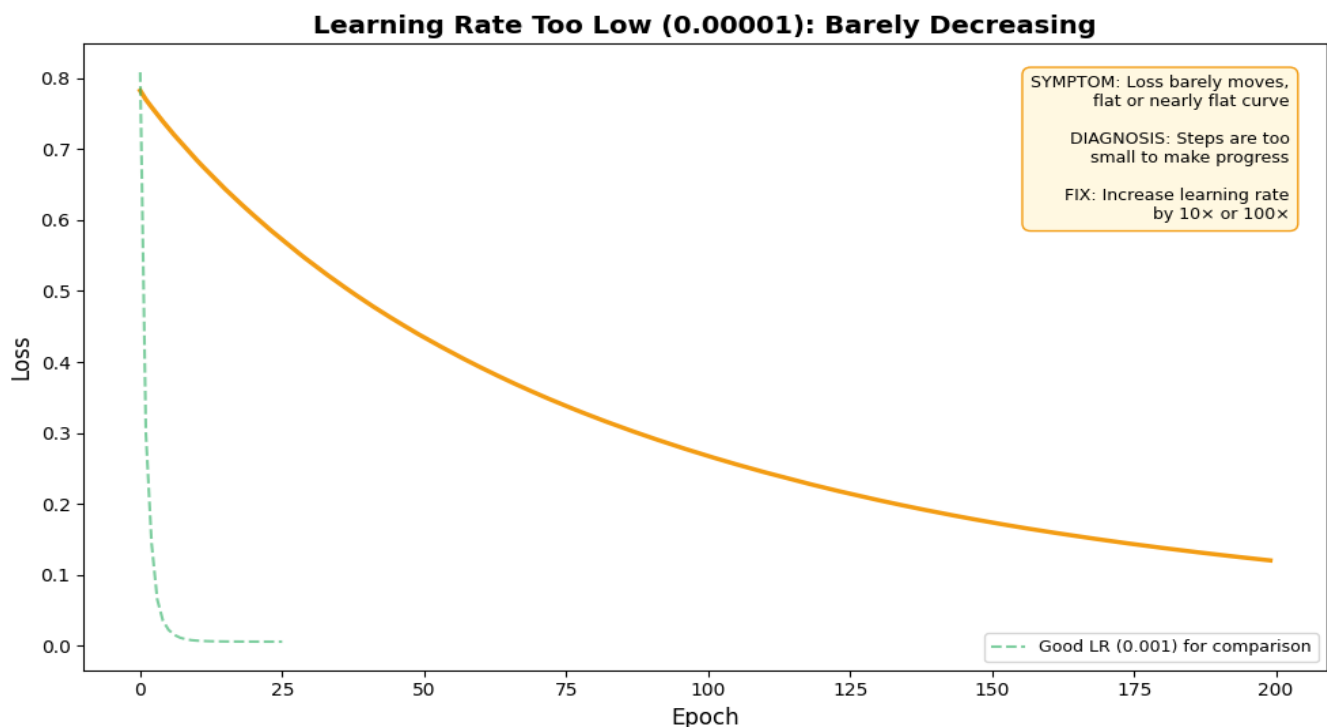


Figure 7. Learning rate too low (0.00001). The loss barely decreases. The green dashed line shows a well-tuned learning rate (0.001) for comparison — much lower loss at the same epoch.

Failure 3: No regularisation on high-dimensional data. Figure 8 shows the most insidious failure. Training accuracy climbs to 100% — the network looks like it is working perfectly. But test accuracy stalls or even declines. The network has memorised the training data, including the noise in 4,000 temporal features. This is the same overfitting from Week 13 (Figure 9), but now severe because 4,000 features give the network far more opportunity to memorise. In Week 13, regularisation barely mattered on 80 features. On 4,000 features, it is essential.

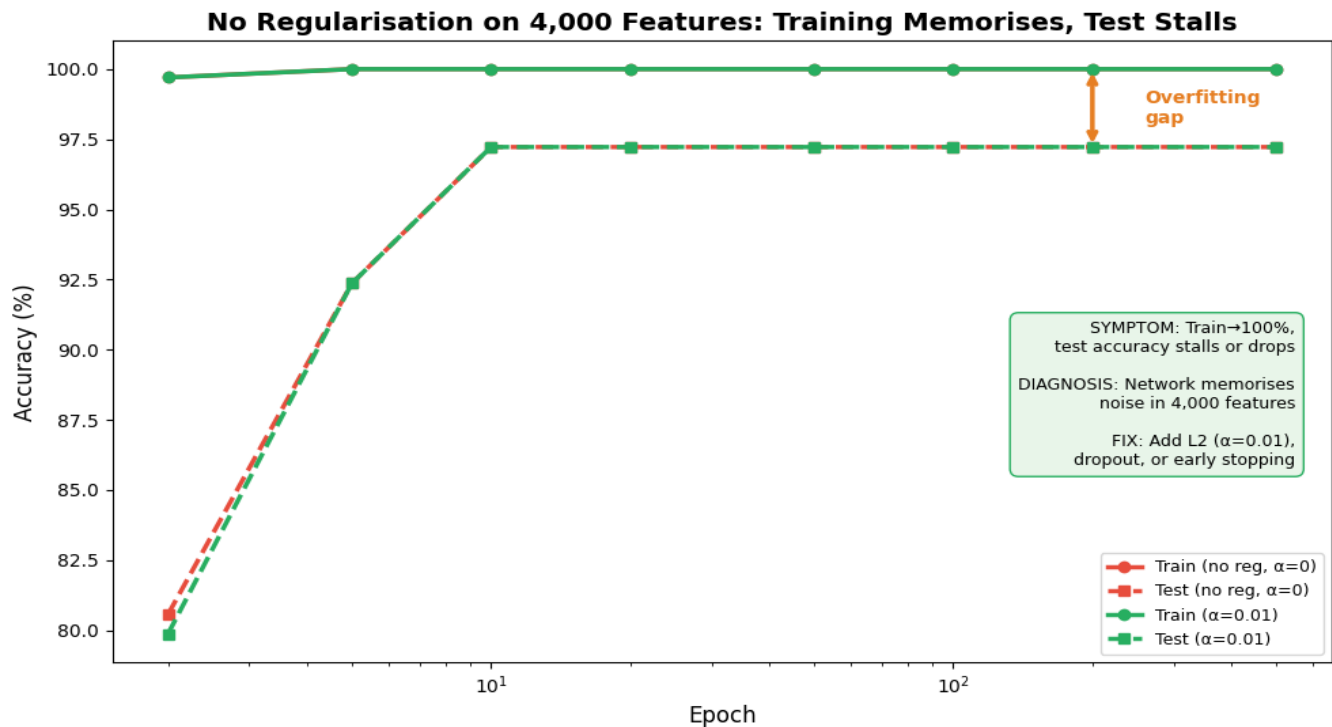


Figure 8. No regularisation on 4,000 features. Red: without regularisation, training reaches 100% but test accuracy stalls (overfitting gap marked). Green: adding L2 regularisation ($\alpha=0.01$) closes the gap — both training and test improve together.

Step 3: Once the loss curve looks healthy, check accuracy. A smooth, steadily decreasing loss curve with a small train/test gap means training is working. Now compare training and validation accuracy: if both are high and close together, the model is learning real patterns. If training is high but validation is low, increase regularisation or reduce model size.

Step 4: Try multiple learning rates. In practice, start with the default learning rate of 0.001 — this is what Adam (the optimizer from Section 3) uses unless you change it — and check the loss curve. If it oscillates, reduce by 10×. If it is flat, increase by 10×. Most problems are solved within two or three adjustments.

5. Data Augmentation for Small Datasets

Can we make 480 trials behave like 4,800?

With 480 trials and 4,000 features, the curse of dimensionality is a real concern. Data augmentation creates synthetic training examples by applying realistic transformations to existing trials. We test three strategies, each with a simple equation.

Gaussian noise. Add random noise to each time bin, simulating trial-to-trial neural variability:

$$\text{rate}_{\tilde{n}}(t) = \text{rate}_n(t) + \varepsilon, \text{ where } \varepsilon \sim \text{Normal}(0, \sigma^2) \text{ and } \sigma = 0.1$$

Each time bin gets an independent random perturbation drawn from a Gaussian with mean zero and standard deviation 0.1. This preserves the overall shape of the temporal profile while adding realistic variability. Result: accuracy rises modestly from 97.5% to 99.2%.

Time jitter. Shift the entire temporal profile by a random number of bins, simulating onset variability:

$$\text{rate}_{\tilde{n}}(t) = \text{rate}_n(t + \Delta t), \text{ where } \Delta t \sim \text{Uniform}(-2, +2) \text{ bins}$$

The profile is shifted left or right by up to 2 bins (20 ms). This is a circular shift — values that fall off one end wrap around to the other. This hurts accuracy (97.5% → 83.9%) because the onset delay IS the impairment signal. Jittering the timing by ± 20 ms obscures exactly the ~ 40 ms delay that the classifier relies on.

Amplitude scaling. Multiply each trial's rates by a random gain factor, simulating recording gain variability across sessions:

$$\text{rate}_{\tilde{n}}(t) = \alpha \times \text{rate}_n(t), \text{ where } \alpha \sim \text{Uniform}(0.8, 1.2)$$

All time bins for all neurons on a given trial are multiplied by the same random factor. This preserves the temporal shape and the onset delay, only changing the overall amplitude. Result: modest improvement (97.5% → 98.6%).

Figure 9 summarises all three. The lesson: augmentation helps when it creates realistic variation that the model needs to handle (noise, gain), but hurts when it destroys the signal (timing jitter on onset-delay data). A researcher must understand their data before choosing an augmentation strategy.

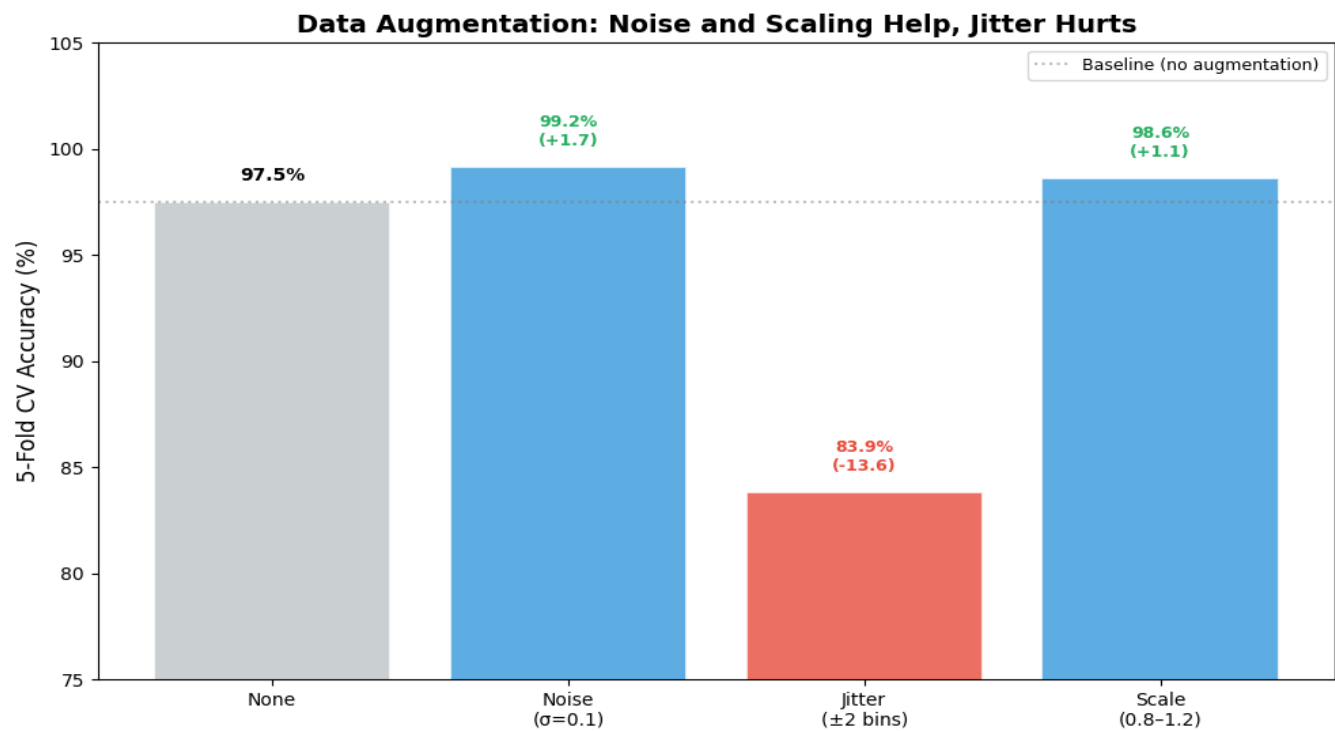


Figure 9. Data augmentation effect on impairment detection (MLP, 5-fold CV). Noise and amplitude scaling (blue) provide modest improvements. Time jitter (red) drops accuracy by 14 points because it obscures the onset delay.

6. Interpreting What the Network Learned

Which neurons and time bins matter to the MLP?

In Section 2, we visualised LR's coefficients — a direct readout of which time bins matter. But the MLP's 7,528 weights are distributed across two hidden layers and do not map directly to input features. How do we interpret what the network has learned?

Saliency maps provide an answer. For each trial, we compute the gradient of the output with respect to each of the 4,000 input features. A large gradient means that a small change in this time bin would change the prediction the most — so the network is “paying attention” to it. We reshape the 4,000-dimensional gradient vector into an 80×50 heatmap (80 neurons $\times$ 50 time bins, where each bin is 10 ms, covering the full 500 ms trial).

Figure 10 compares the LR coefficients with the MLP saliency map. Panel A shows the same LR coefficients as Figure 4, but now as a full 80×50 heatmap instead of averaged across neurons — so you can see which specific neurons at which specific times matter most. (Figure 4 was the average of each column in this heatmap.) Panel B shows the MLP saliency.

On our simulated data, both panels look similar — they highlight the same onset window. This is expected: we generated the onset delay uniformly across all 80 neurons, so there are no neuron-specific temporal patterns for the MLP to discover that LR cannot. The value of this figure is the technique, not the result on this particular dataset. On real cortical recordings, where impairment might affect some neurons more than others — for example, neurons in one cortical column might show a larger onset delay than neurons in another — the MLP saliency map would diverge from LR's more uniform coefficients, revealing neuron-time interactions that a linear model cannot capture.

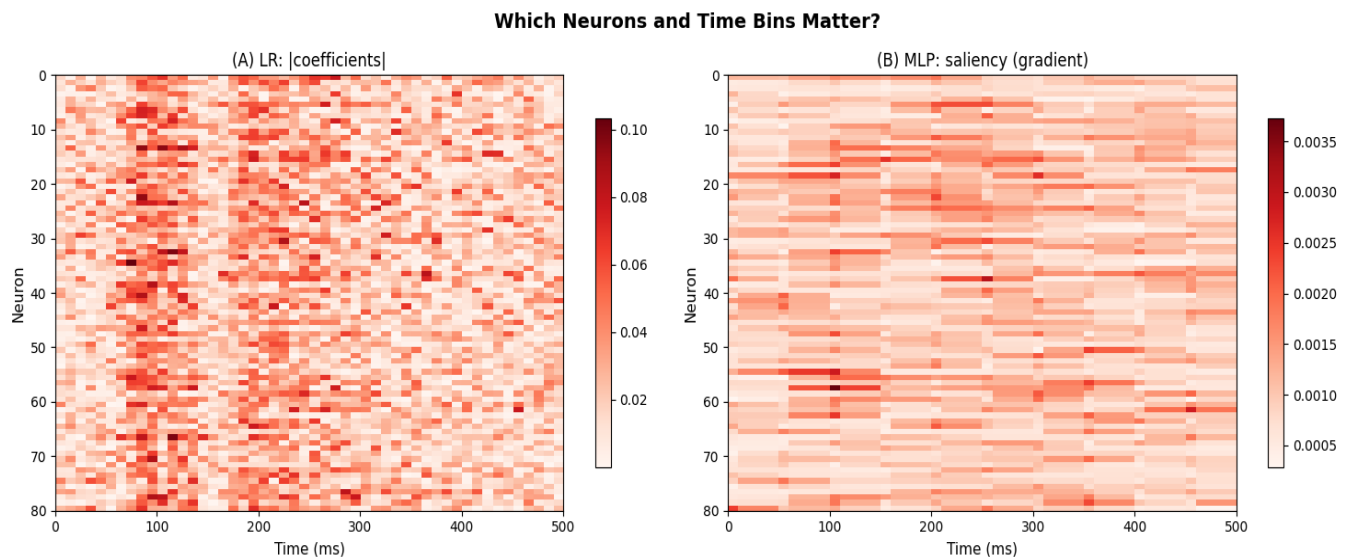


Figure 10. Which neurons and time bins matter? (A) LR: absolute coefficients reshaped to 80 neurons $\times$ 50 time bins. (B) MLP: saliency map (gradient magnitude). On this simulated data, both highlight the same onset window. On real data with neuron-specific impairment effects, the MLP saliency would reveal patterns that LR cannot.

Saliency maps are the neural network analogue of the interpretability tools from previous weeks: LR coefficients (Week 5), LDA loadings (Week 10), and RF permutation importance (Week 11). The difference is resolution: those tools identified important neurons; saliency maps identify

important neuron-time combinations.

7. Drift Robustness on Temporal Features

Do temporal features survive electrode drift?

Continuing the drift analysis from Weeks 9–13, we add Gaussian noise to the temporal features and measure how each method degrades on impairment detection. Temporal features have 50× more dimensions than mean rates (4,000 vs 80), which could make drift worse — or the redundancy across time bins could provide natural tolerance.

Figure 11 shows the results. LR is the most drift-robust method (98.8% → 94.4% at $\sigma=10$), losing only 4 points. This makes sense: LR assigns a separate weight to each of 4,000 time bins, so drift in one bin is compensated by the other 3,999. MLP is second (97.5% → 90.6%), again benefiting from distributed representations across its hidden layer — the same mechanism we saw in Week 13 (Figure 13) on mean rates. RF drops sharply (91.9% → 72.1%) because its axis-aligned splits on 4,000 noisy dimensions are brittle.

The drift results reinforce the week’s central lesson: on temporal features, LR’s simplicity is a strength, not a weakness. It is more accurate (99.0% vs 96.9%), more interpretable (Figure 4), and more drift-robust (94.4% vs 90.6% at $\sigma=10$) than the MLP. The MLP’s value lies elsewhere — on problems where the signal is nonlinear and a simple weighted sum of time bins is not enough.

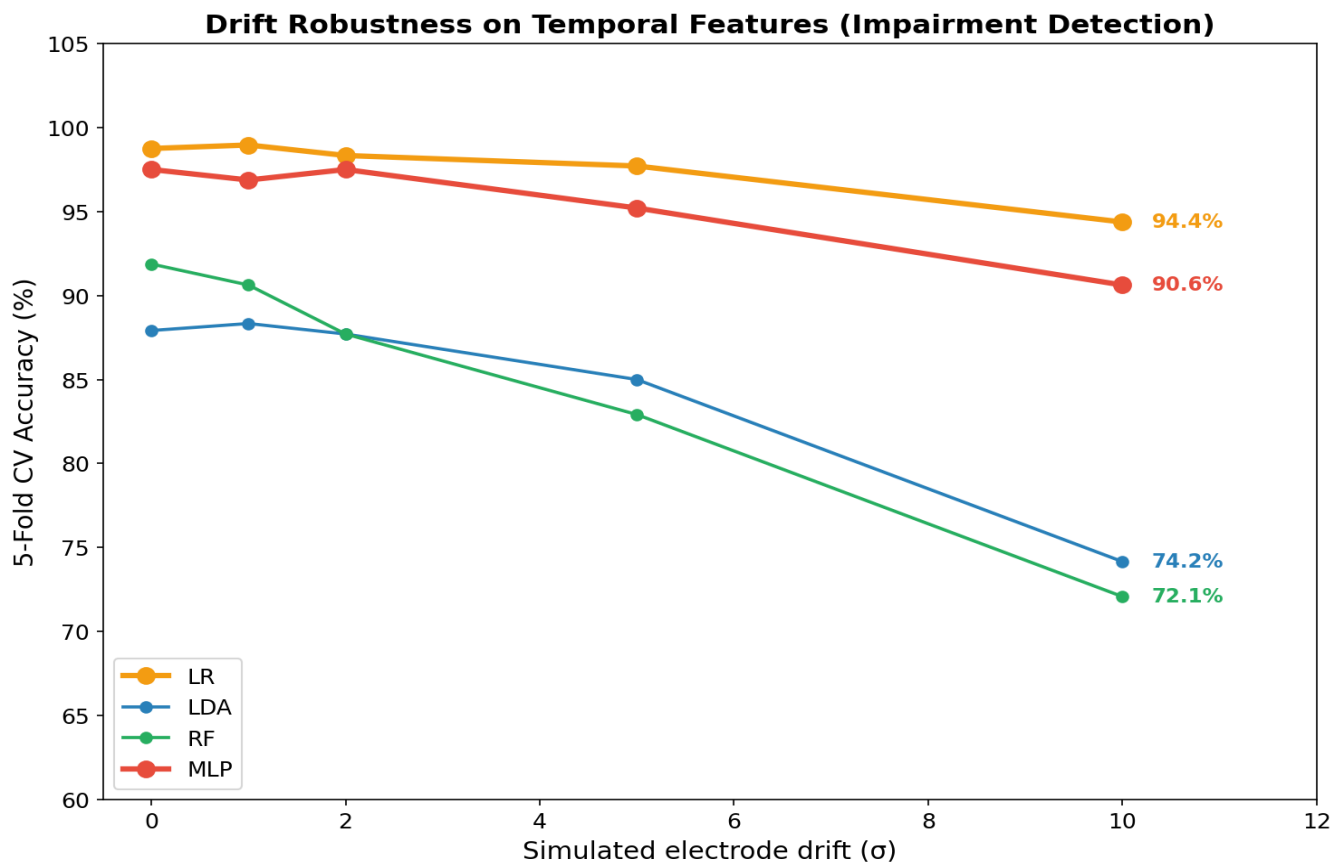


Figure 11. Drift robustness on temporal features (impairment detection, 5-fold CV). LR (orange) degrades most slowly, followed by MLP (red). RF (green) drops sharply. On temporal data, LR’s 4,000 individual weights provide natural redundancy against per-neuron drift.

8. The Decision Framework

Given a new dataset, how do you choose a method?

The course has now covered 10 methods across 14 weeks. Figure 12 distils the decision process into a flowchart that students can use in their own research.

The first question is not “which classifier?” but “what features?” This week’s central result — LR on temporal features (99.0%) beating RF on mean rates (86.5%) by 13 points — proves that feature design dominates classifier choice. Before reaching for a complex model, ask whether richer features would solve the problem with a simpler one.

The Decision Framework: Choosing the Right Method

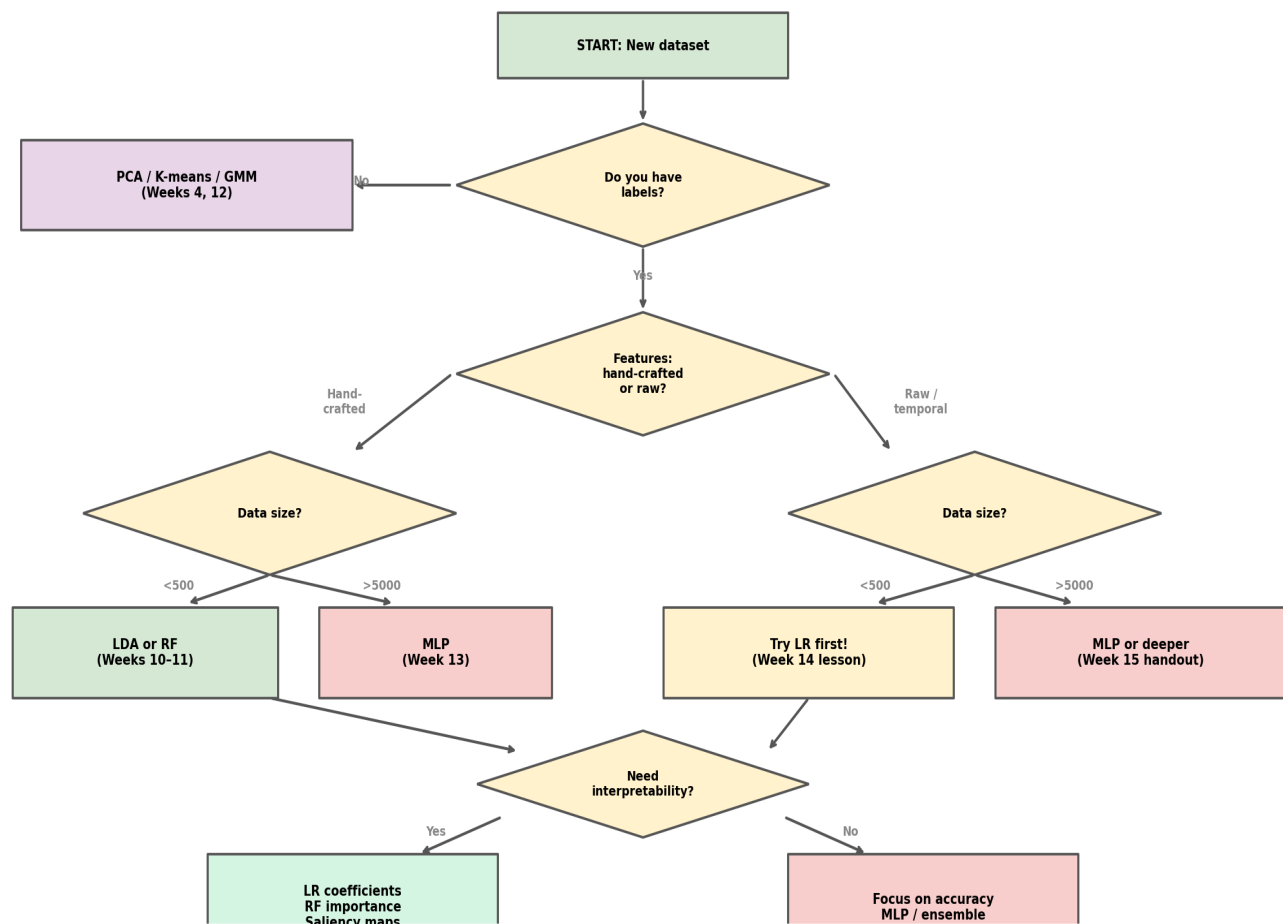


Figure 12. The decision framework. Start with the data: do you have labels? What kind of features? How much data? Each path leads to a recommended method, with interpretability as a secondary consideration.

Key takeaways from the flowchart:

- No labels → PCA, k-means, or GMM (Week 4 and 12). You cannot use supervised methods without ground truth.
- Hand-crafted features, small data (<500 trials) → LDA or RF. They match or beat MLPs with less effort and better interpretability (Weeks 10–11, confirmed in Week 13).

- Raw/temporal features, small data → try LR first. It may surprise you (99.0% this week). Then try MLP if LR's linear assumption fails.
- Raw features, large data (>5,000 trials) → MLP, or deeper architectures described in the Week 15 handout.
- Need interpretability → LR coefficients, RF importance, or saliency maps. All three are available regardless of classifier choice.
- Need drift robustness → On mean rates, MLP's distributed representations provide natural tolerance (Week 13, Figure 13). On temporal features, LR is actually the most robust (Figure 11) because 4,000 individual weights provide redundancy.

What we learned this week

- Temporal features transform impairment detection: from ~70–85% (mean rates) to ~88–99% (temporal). The information we discarded for 13 weeks was exactly what we needed.
- Features matter more than the classifier. LR on temporal features (99.0%) beat MLP on mean rates (79.8%) by 19 points.
- Regularisation that did not matter on 80D (Week 13) becomes essential on 4,000D. The answer is always “it depends on your data.”
- Data augmentation helps when it creates realistic variation, but hurts when it destroys the signal (time jitter on onset-delay data).
- Saliency maps extend interpretability to neural networks: neuron-by-time-bin resolution, compared with per-neuron resolution from RF and LR.

References

Paszke A, Gross S, Massa F, et al. PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems* 32, 2019.

Simonyan K, Vedaldi A, Zisserman A. Deep inside convolutional networks: Visualising image classification models and saliency maps. *arXiv:1312.6034*, 2013.

Shorten C, Khoshgoftaar TM. A survey on image data augmentation for deep learning. *Journal of Big Data*, 6(1):60, 2019.

Glaser JI, Benjamin AS, Farhoodi R, Kording KP. The roles of supervised machine learning in systems neuroscience. *Progress in Neurobiology*, 175, 126–137, 2019.

Georgopoulos AP, Kalaska JF, Caminiti R, Massey JT. On the relations between the direction of two-dimensional arm movements and cell discharge in primate motor cortex. *Journal of Neuroscience*, 2(11), 1527–1537, 1982.